

SOAR Deployment Documentation

Wazuh – Suricata – TheHive – Cortex – MISP Integration

Security Operations Center Automation & Threat Intelligence Enrichment

Overall SOC Workflow

The end-to-end workflow below describes how an attack is detected, escalated, enriched, and closed within the integrated SOAR environment.

- **Step 1:** Kali Linux Attacker generates an attack (ICMP Flood, Port Scan, SSH Brute Force) against the Ubuntu victim server.
- **Step 2:** Suricata IDS, running on the Ubuntu server, detects the malicious traffic and writes an alert to `/var/log/suricata/eve.json`.
- **Step 3:** The Wazuh Agent reads `eve.json` and forwards the event to the Wazuh Manager (SIEM).
- **Step 4:** The Wazuh Manager decodes the log, applies detection rules, assigns severity, and displays the alert on the Wazuh Dashboard.
- **Step 5:** A custom Wazuh integration forwards the alert to TheHive via its REST API.
- **Step 6:** TheHive automatically creates an Incident Case and extracts observables (IP, URL, hash, domain, email, filename).
- **Step 7:** The analyst opens an observable and runs Cortex Analyzers (VirusTotal, MISP, and others).
- **Step 8:** Cortex queries external threat-intelligence sources and returns enrichment results — reputation, IOC context, malware information — to TheHive.
- **Step 9:** The SOC analyst reviews the enriched case and classifies it as a True Positive, False Positive, or Benign Activity.
- **Step 10:** The analyst proceeds through Containment → Eradication → Recovery, and the incident is closed.

Network Architecture

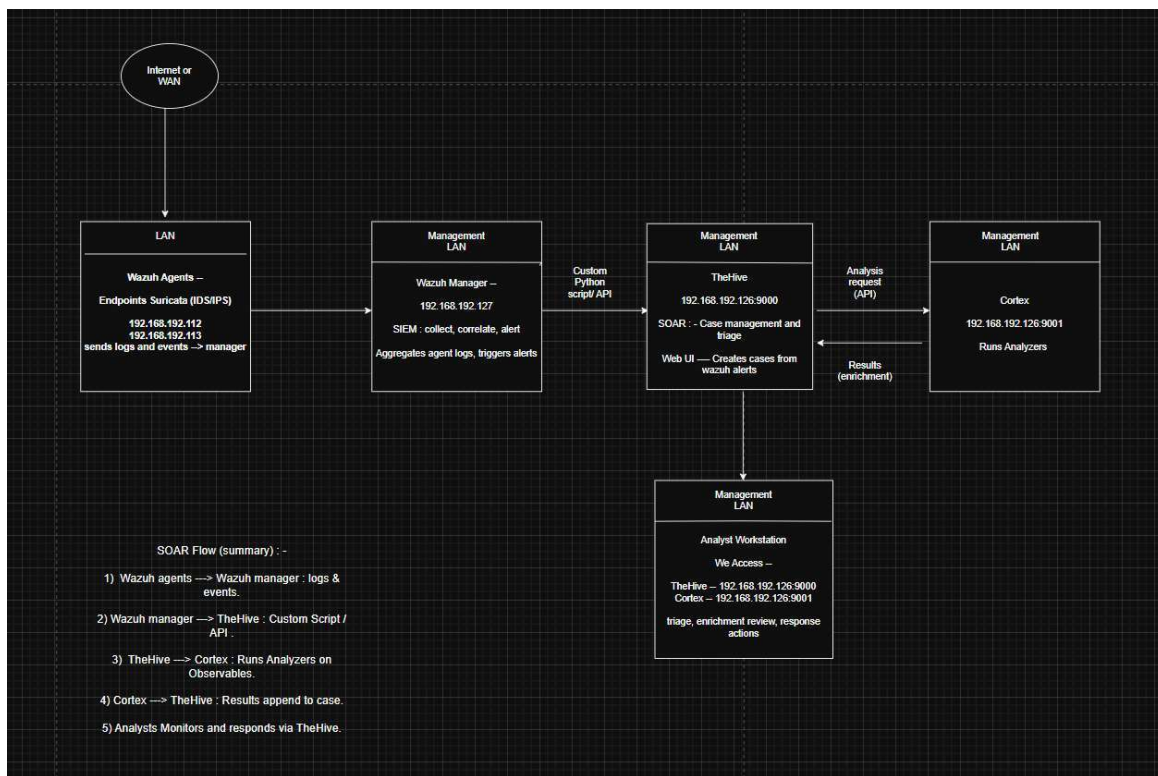


Figure A — SOAR network architecture and data-flow overview.

Wazuh Agents on monitored endpoints run Suricata (IDS/IPS) and forward logs and events to the Wazuh Manager over the management LAN. The Wazuh Manager (SIEM) aggregates agent logs, correlates them, and triggers alerts, which are pushed to TheHive via a custom Python script/API. TheHive performs case management and triage, and requests analysis from Cortex over its API; Cortex runs the configured analyzers and returns enrichment results to TheHive. Analysts access both TheHive and Cortex from an analyst workstation for triage, enrichment review, and response actions.

Overall Flow of Alert to Case in the SOAR Setup :

1. Wazuh Agent (Detection Layer)

Purpose: Collects security logs from monitored endpoints.

- The Wazuh Agent continuously monitors endpoints for suspicious activities (e.g., ICMP Flood, SSH brute force, file integrity changes, authentication logs).
- When a security event matches a detection rule, the agent forwards the event to the Wazuh Manager.

2. Wazuh Manager (Alert Generation & Correlation)

Purpose: Analyzes collected logs and generates security alerts.

- The Wazuh Manager parses incoming logs from agents.
- It applies decoders, rules, and correlation logic.
- A rule-based alert is generated based on the detected activity.

The alert (JSON format) is forwarded to the custom integration script located in `/var/ossec/integrations/`

3. Custom Python Integration Script (Automation Bridge)

Purpose: Automatically transfers Wazuh alerts into TheHive.

- The integration script receives the Wazuh alert.
- It uses the TheHive REST API to automatically create a new case or alert.

- Information such as Source IP, Destination IP, Rule ID, Severity, Timestamp, and Alert description is included in the created incident.

4. TheHive (Case Management Platform)

Purpose: Centralized incident and case management.

- TheHive receives alerts from Wazuh and creates a structured investigation case.
- Important observables are extracted automatically, such as IP Address, Domain, URL, Hashes, and Email Address.
- SOC analysts can assign cases, add tasks, collaborate with team members, and document investigation findings.

5. Cortex (Analysis & Enrichment Engine)

Purpose: Executes analyzers to enrich observables with threat intelligence.

- Analysts select an observable inside TheHive, and TheHive sends the observable to Cortex.
- Cortex runs one or more analyzers, such as VirusTotal, the MISP Analyzer, AbuseIPDB, URLhaus, WHOIS, and GeoIP.
- Cortex collects intelligence from external or internal sources.

6. MISP (Threat Intelligence Platform)

Purpose: Stores and provides Threat Intelligence (IOCs).

- When the MISP Analyzer is executed, Cortex sends the observable (IP, Hash, Domain, URL, etc.) to MISP.
- MISP searches its IOC database. If a matching IOC is found, MISP returns information such as Event ID, Event Description, Threat Type, Malware Family, Related Attributes, Tags, and Known Campaigns.

```
Observable
↓
192.168.0.7
↓
Cortex MISP Analyzer
↓
MISP Database
↓
Event 6016
"DDoS Attack targeting internal infrastructure"
```

7. Cortex (Returns Enrichment Results)

Purpose: Formats analyzer results and sends them back to TheHive.

- Cortex receives responses from MISP, VirusTotal, AbuseIPDB, and other analyzers.
- It combines the results into a standardized report, which is returned to TheHive.

8. TheHive (Enriched Investigation Case)

Purpose: Displays all enrichment results within the same incident.

- TheHive automatically attaches the analyzer reports.
- Analysts can immediately view VirusTotal reputation, MISP event matches, AbuseIPDB score, WHOIS information, and GeoIP details — providing complete investigation context without leaving TheHive.

9. SOC Analyst (Investigation & Response)

Purpose: Investigates, validates, and responds to incidents.

- Using all collected information, the analyst reviews the original Wazuh alert and correlates related events.
- The analyst examines threat intelligence and determines whether the alert is malicious or benign.
- The analyst performs containment and remediation actions, documents findings, and closes or escalates the case if necessary.

Purpose of Each Component :

Component	Purpose
Wazuh Agent	Collects endpoint logs and forwards them to the Wazuh Manager.
Wazuh Manager	Parses logs, applies detection rules, correlates events, and generates security alerts.
Python Integration Script	Automates the transfer of Wazuh alerts to TheHive using the REST API.
TheHive	Manages security incidents, investigations, cases, tasks, observables, and analyst collaboration.
Cortex	Executes analyzers and responders to enrich observables with threat intelligence.
MISP	Maintains an internal repository of Indicators of Compromise (IOCs) and provides threat intelligence lookups.
VirusTotal	Supplies external reputation data for files, IP addresses, URLs, and domains using multiple antivirus engines.
SOC Analyst	Investigates alerts, validates threats, performs incident response, documents findings, and coordinates remediation.

Environment Verification :

Before configuring the integration, it is essential to verify the versions of the operating system, Wazuh, TheHive, and Python to ensure compatibility. This also helps future maintenance and replication of the same environment.

```

kiran@soarhive:~$ sudo docker ps
CONTAINER ID   IMAGE                                NAMES                                COMMAND                                CREATED        STATUS        PORTS
8ba3daf20aa7   strangebee/thehive:5.2              thehive-cortex-thehive-1            "/opt/thehive/entryp..." 38 hours ago   Up 13 hours   0.0.0.0:9000->9000/tc
8c0edcc7fb71   thehiveproject/cortex:latest        thehive-cortex-cortex.local-1       "/opt/cortex/entrypo..." 38 hours ago   Up 13 hours   0.0.0.0:9001->9001/tc
da3fab504558   coolacid/misp-docker:core-latest    thehive-cortex-misp.local-1         "/entrypoint.sh"         38 hours ago   Up 13 hours   0.0.0.0:80->80/tcp, 0
0.0.0.0:443->443/tcp
2b5f23713bdd   cassandra:4                         thehive-cortex-cassandra-1          "docker-entrypoint.s..." 38 hours ago   Up 13 hours   7000-7001/tcp, 7199/t
cp, 9160/tcp, 0.0.0.0:9042->9042/tcp
thehive-cortex-elasticsearch-1       "/bin/tini -- /usr/l..." 38 hours ago   Up 13 hours   0.0.0.0:9200->9200/tc
479f08b736b4   docker.elastic.co/elasticsearch:7.17.9
thehive-cortex-minio-1               "/usr/bin/docker-ent..." 38 hours ago   Up 13 hours   9000/tcp, 0.0.0.0:900
0.0.0.0:3000->3000/tcp
2->9002/tcp
fac12e21c159   mysql/mysql-server:5.7              thehive-cortex-misp-mysql-1         "/entrypoint.sh mysql..." 38 hours ago   Up 13 hours (healthy) 3306/tcp, 33060/tcp

kiran@soarhive:~$ sudo docker inspect -f '{{.Config.Image}}' thehive-cortex-cortex.local-1
thehiveproject/cortex:latest
kiran@soarhive:~$ sudo docker inspect -f '{{.Config.Image}}' thehive-cortex-thehive-1
strangebee/thehive:5.2

```

Figure 1 — Environment verification: Ubuntu, Wazuh Manager, TheHive, and Python versions used in the deployment, confirming compatibility across components.

TheHive Installation & Verification :

- TheHive was installed and configured as the Security Operations and Incident Response platform.
- After installation, the service was verified to ensure it runs on the default port 9000.
- TheHive provides an intuitive web interface for analysts to manage security incidents as cases, link observables, and correlate events with MITRE ATT&CK techniques.
- Once logged into the web interface, the administrator can generate an API key (under Profile → API keys).
- This API key is crucial because it allows the Wazuh Manager integration script to authenticate securely with TheHive API for automated case creation.

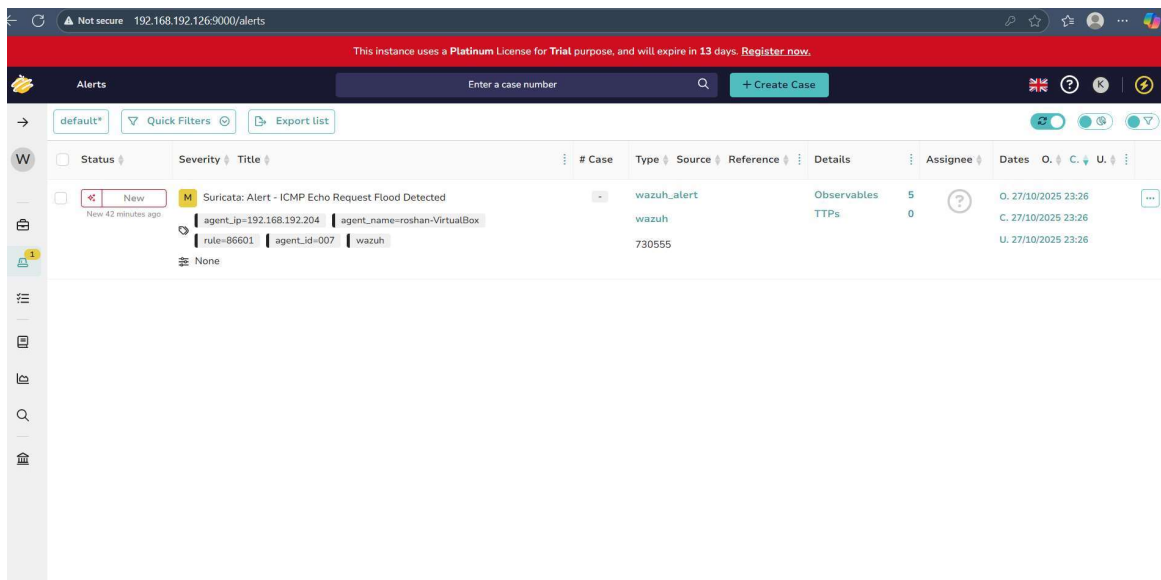


Figure 2 — TheHive successfully deployed. The interface confirms TheHive is running on port 9000 and ready to receive API requests from Wazuh.

Wazuh – TheHive Integration :

- Wazuh is configured to automatically forward alerts to TheHive using a custom integration script.
- The integration is placed under `/var/ossec/integrations/`, where Wazuh looks for executable scripts matching the names defined in the configuration file.
- The integration consists of two parts:
 - A Bash wrapper (`custom-w2thive`) that Wazuh executes directly.
 - A Python script (`custom-w2thive.py`) that uses `thehive4py` to send alerts to TheHive's API endpoint.
- Proper permissions (`chmod 755`) and ownership (`root:wazuh` or `root:ossec`) are assigned so that Wazuh can execute the files without permission errors.

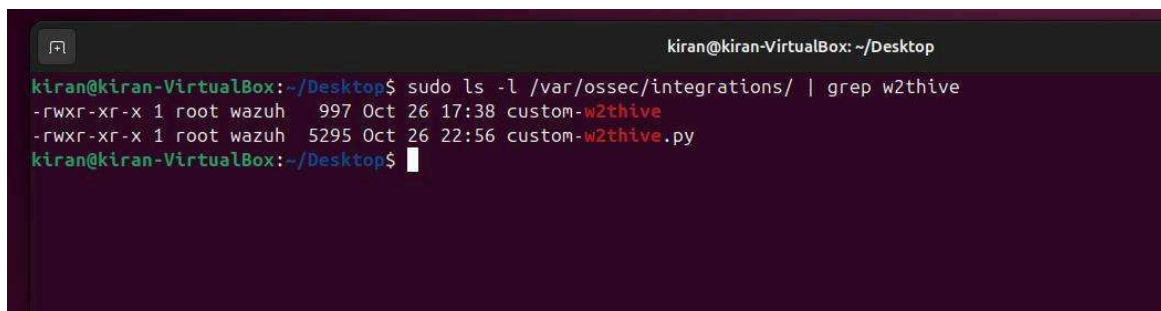


Figure 3 — Custom integration scripts in Wazuh. `custom-w2thive` (Bash wrapper) and `custom-w2thive.py` (Python script) are present and executable under `/var/ossec/integrations/`, handling communication with TheHive via the `thehive4py` API library.



Figure 4 — Wazuh Manager integration configuration. The integration passes the alert JSON to the custom script for forwarding to TheHive.

```
#start user config

# Global vars

#threshold for wazuh rules level
lvl_threshold=5
#threshold for suricata rules level
suricata_lvl_threshold=3
```

Figure 5 — Wazuh alert-level thresholds. This snippet instructs Wazuh to trigger the custom-w2thive integration whenever a Wazuh alert of level ≥ 5 , or a Suricata alert of level ≥ 3 , is generated.

Alert Generation and Forwarding :

- After the integration setup, the next step is to verify the flow of alerts from Wazuh to TheHive.
- When a detection rule is triggered — for example, during an ICMP flood or SSH brute-force test — Wazuh generates an alert in JSON format and passes it to the configured integration.
- The Bash wrapper captures this alert and calls the Python script, which then sends it to TheHive via REST API.
- The alert JSON contains all important details such as rule.id, rule.description, srcip, and agent.name, which are used to create a meaningful case title and description in TheHive.

```
roshan@roshan-VirtualBox: ~/Desktop
{
  "flow": {
    "pkts_toserver": 135401,
    "pkts_toclient": 134410,
    "bytes_toserver": 8124060,
    "bytes_toclient": 5645220,
    "start": "2025-10-08T22:02:18.577312+0530",
    "src_ip": "192.168.192.107",
    "dest_ip": "192.168.192.204"
  }
}
{
  "timestamp": "2025-10-08T22:04:31.496501+0530",
  "flow_id": 790687442347598,
  "to_iface": "enp0s3",
  "event_type": "alert",
  "src_ip": "192.168.192.107",
  "dest_ip": "192.168.192.204",
  "proto": "ICMP",
  "ip_v": 4,
  "icmp_type": 8,
  "icmp_code": 0,
  "pkt_src": "wire/pcap",
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 1000005,
    "rev": 1,
    "signature": "ICMP Echo Request Flood Detected",
    "category": "",
    "severity": 3
  },
  "direction": "to_server",
  "flow": {
    "pkts_toserver": 135402,
    "pkts_toclient": 134410,
    "bytes_toserver": 8124120,
    "bytes_toclient": 5645220,
    "start": "2025-10-08T22:02:18.577312+0530",
    "src_ip": "192.168.192.107",
    "dest_ip": "192.168.192.204"
  }
}
```

Figure 6a — Example Wazuh alert triggered by a test attack. The ICMP flood detection alert generated by Wazuh includes key fields such as rule.id, rule.description, agent.name, and srcip, which are forwarded to TheHive through the integration script.

TheHive Automatic Case Creation Result :

- When TheHive receives the alert through its API, it automatically creates a new case based on the data provided by the integration script.
- Each case in TheHive includes:
 - A title (derived from the Wazuh alert description).

- A description (the complete alert JSON).
- Tags for identification (e.g., wazuh, icmp_flood, ssh_bruteforce).
- Optionally, observables extracted from the alert (such as source IP or file hash).
- This automated case creation streamlines the workflow — security analysts can directly open TheHive, review alert details, add comments, and take response actions without manually checking Wazuh logs.

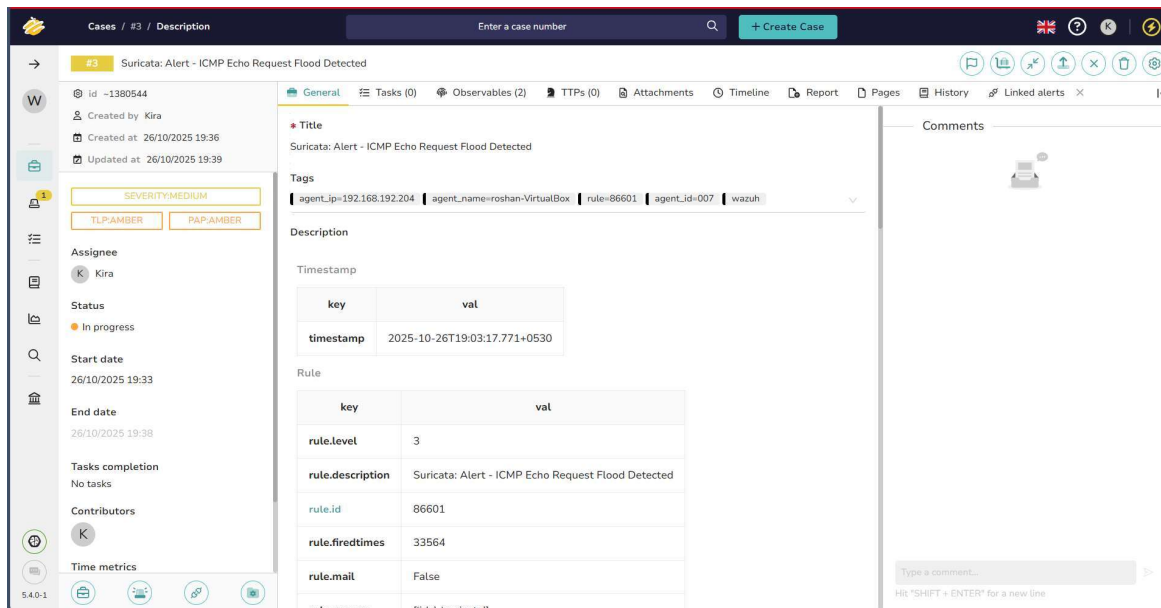


Figure 6b — Case automatically created in TheHive. This confirms successful end-to-end integration: Wazuh detected the ICMP flood attack, forwarded the alert, and TheHive created a case with the full alert JSON in its description. Analysts can now triage, assign, and enrich the case directly within TheHive.

MITRE ATT&CK Mapping :

- TheHive allows analysts to enrich and categorize cases according to the MITRE ATT&CK framework.
- By linking alerts or cases to specific ATT&CK techniques (e.g., T1498 – Network Denial of Service), SOC teams can quickly assess the adversary tactics involved and prioritize response efforts.
- During the integration process, MITRE technique identifiers can be added either automatically by the Python script (via a mapping file of rule IDs to techniques) or manually by analysts within the TheHive UI.

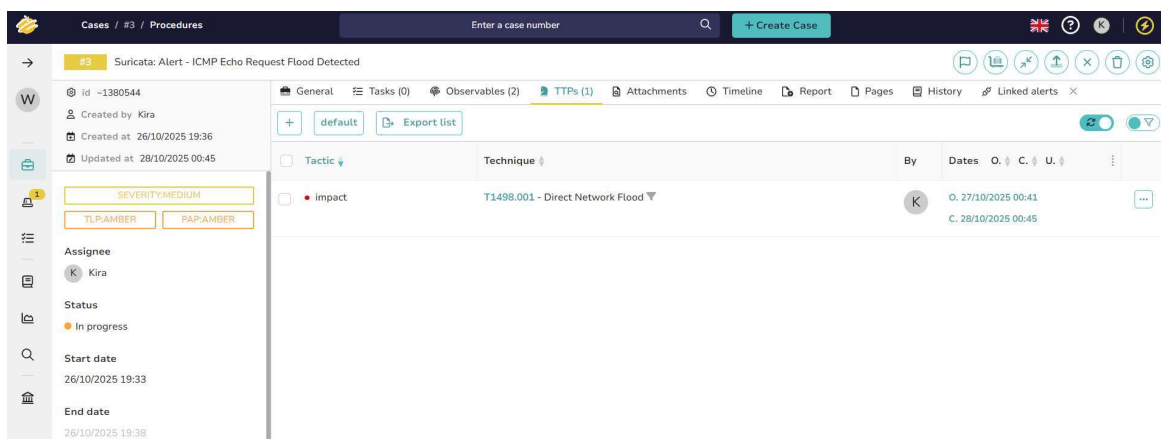


Figure 7 — Mapping the observable extracted from Wazuh to techniques in the MITRE ATT&CK framework (T1498.001 – Direct Network Flood).

Cortex Integration and Analyzer Configuration

- Cortex is an open-source analysis and response engine designed to work alongside TheHive. It allows analysts to run hundreds of analyzers and responders on observables, such as IP addresses, domain names, file hashes, or URLs.
- When integrated with TheHive, Cortex automates threat enrichment and incident validation, reducing the manual effort needed by SOC analysts during investigations.
- In this setup, TheHive sends observables extracted from cases to Cortex, which performs external lookups and returns enriched intelligence results. This workflow helps analysts rapidly determine if an observable is malicious and provides context for decision-making.

Cortex Web Interface Access :

- Cortex is hosted on the same server as TheHive (192.168.192.126) and operates on port 9001.
- Accessing the Cortex web interface confirms that the service is running properly and available for integration.
- From here, administrators can manage analyzers, responders, API keys, and organization settings.

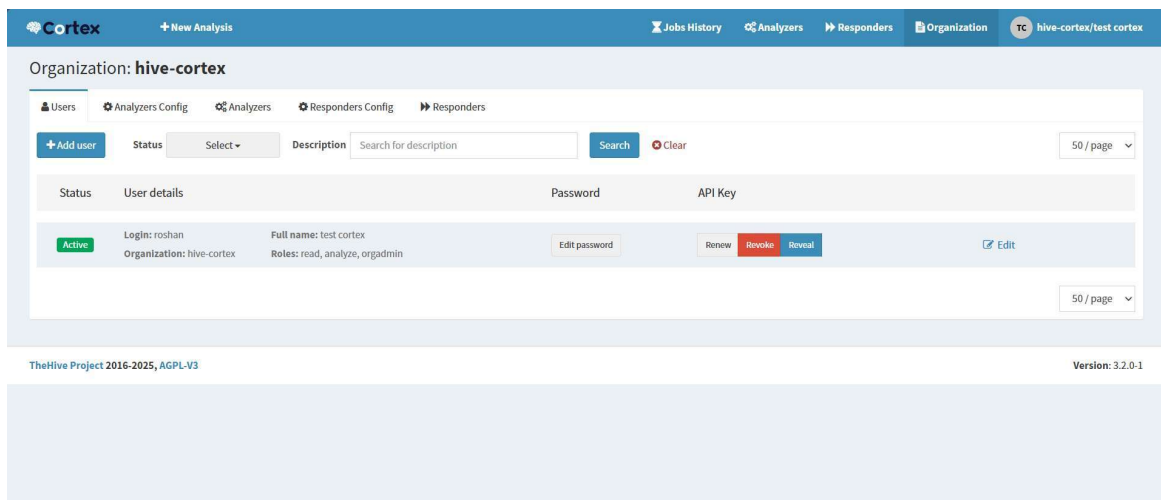


Figure 8 — Cortex web interface running on port 9001. This verifies that the Cortex service is successfully deployed and accessible on the network, forming the core of automated analysis and response capabilities in the SOAR environment.

TheHive – Cortex Connection Verification

- Once Cortex is running, TheHive must be configured to communicate with it through the API endpoint.
- The integration is achieved by adding the Cortex instance details under TheHive's administration settings and validating the connection.
- A successful connection ensures that TheHive can trigger Cortex analyzers when analysts request enrichment on observables.

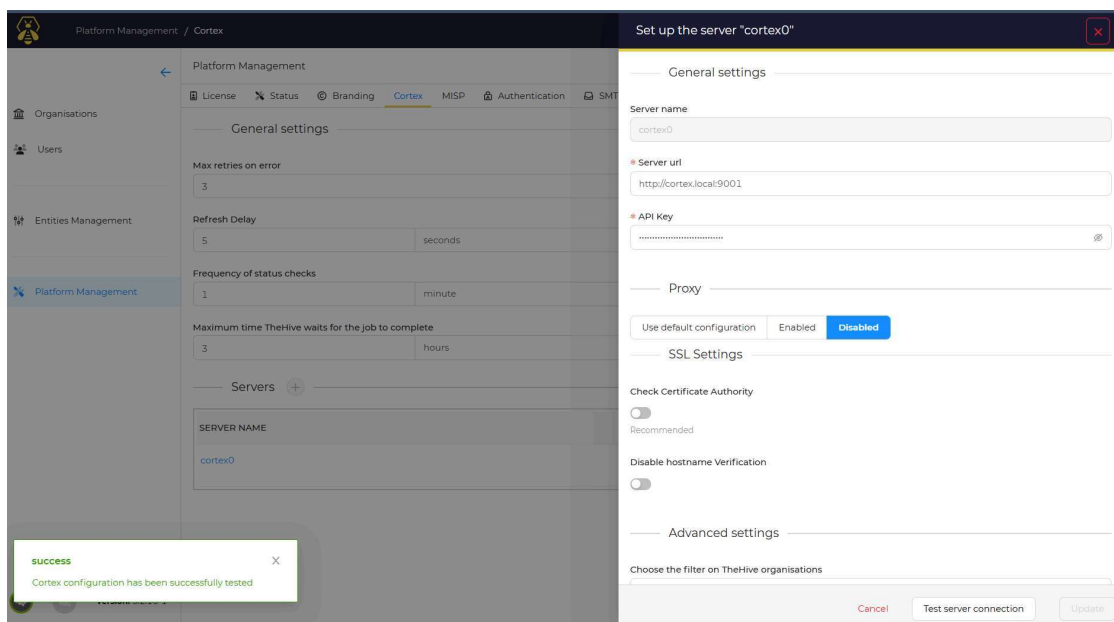


Figure 9 — TheHive–Cortex integration test. TheHive connects to the Cortex instance at 192.168.192.126:9001 using the configured API key.

Analyzer Configuration :

- Analyzers are modular plugins in Cortex that interact with third-party threat intelligence services. Each analyzer requires an API key or credentials, configured in its config.json file under /opt/cortex/analyzers/.
- For example, the VirusTotal_GetReport_3_1 analyzer requires a VirusTotal API key. Once configured, Cortex can query these external services and send back detailed results to TheHive.

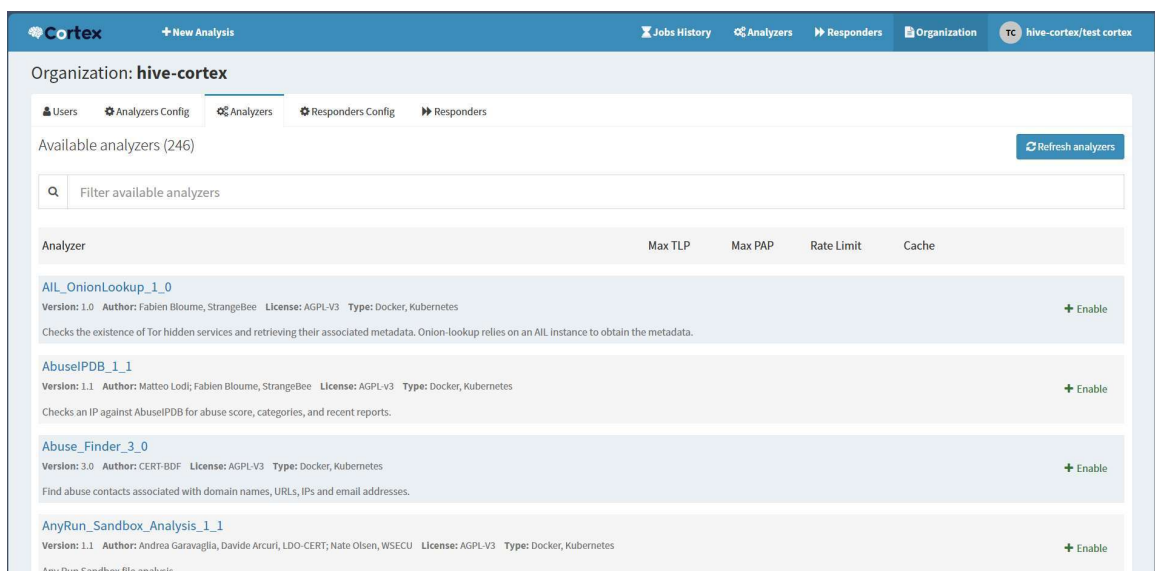


Figure 10 — Available analyzers list in Cortex.

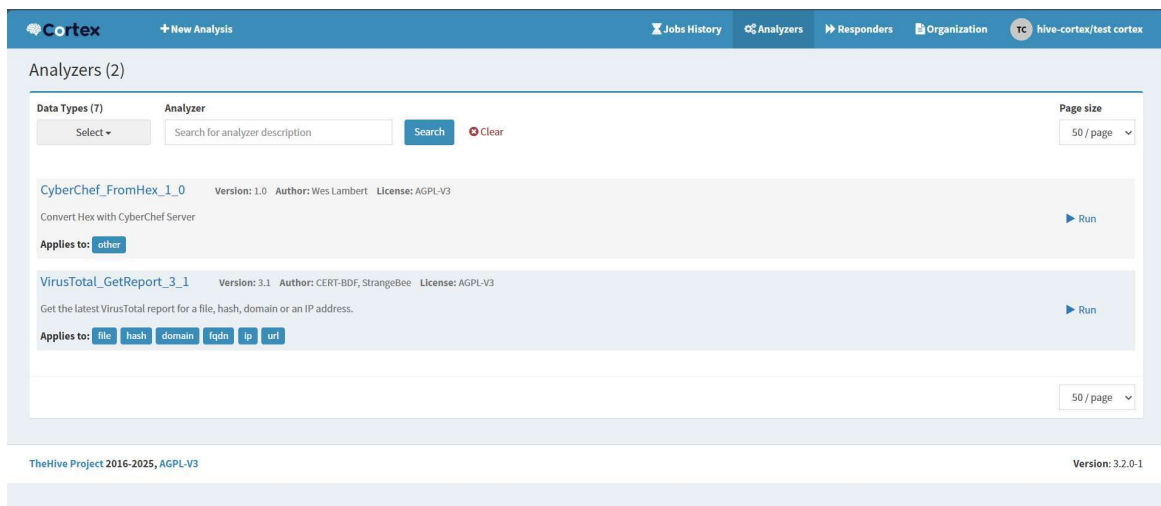


Figure 11 — Active analyzers enabled in the organization. The screenshot displays the analyzers available for use on the Cortex platform.

Analyzer Execution Result

- Once analyzers are properly configured, analysts can execute them from within a case. For example, selecting an observable such as an IP address and running the VirusTotal analyzer triggers Cortex to fetch reputation data.
- TheHive then displays this information in the case as part of the enrichment results, giving immediate context for triage.

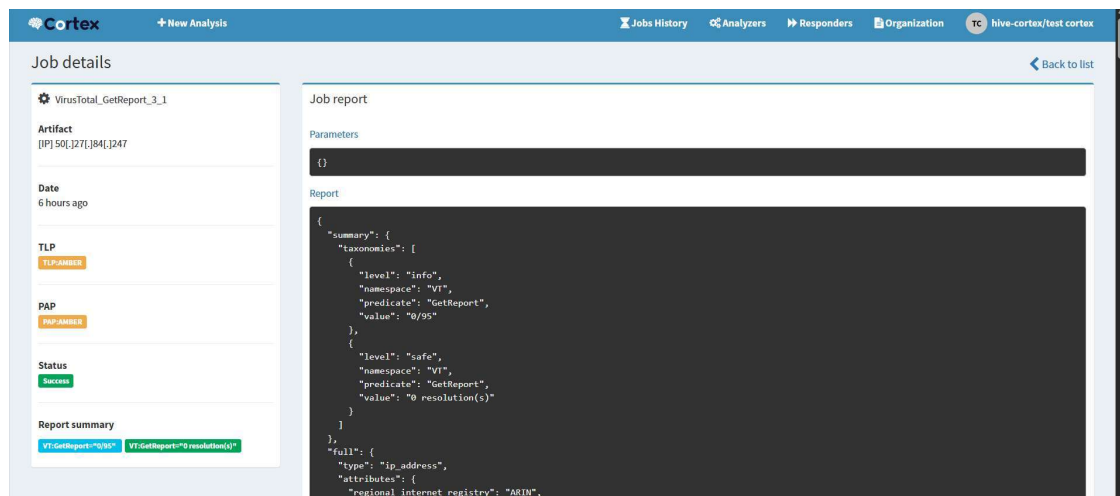


Figure 12a — Cortex analyzer execution and result. Cortex retrieved threat data from external intelligence feeds, providing analysts with actionable insights for further investigation.

Cortex Job History

- Cortex maintains a record of all executed jobs, including their execution status and result data. This helps administrators monitor analyzer performance. If a job fails, error logs in this section can be used for troubleshooting configuration or connectivity issues.

Status	Job details	TLP	PAP
Success	[ip] 50[.]27[.]84[.]247 Analyzer: VirusTotal_GetReport_3_1 Date: 6 hours ago User: hive-cortex/roshan	TLP-AMBER	PAP-AMBER
Success	[ip] 192[.]168[.]192[.]206 Analyzer: VirusTotal_GetReport_3_1 Date: 6 hours ago User: hive-cortex/roshan	TLP-AMBER	PAP-AMBER
Success	[ip] 192[.]168[.]192[.]204 Analyzer: VirusTotal_GetReport_3_1 Date: 6 hours ago User: hive-cortex/roshan	TLP-AMBER	PAP-AMBER
Success	[hash] ffb055b6aea06076f2e25772f3f0655e Analyzer: VirusTotal_GetReport_3_1 Date: 7 hours ago User: hive-cortex/roshan	TLP-AMBER	PAP-AMBER
Success	[hash] 6a8401448a5bd2b540850f811b20a66d Analyzer: VirusTotal_GetReport_3_1 Date: 7 hours ago User: hive-cortex/roshan	TLP-AMBER	PAP-AMBER

Figure 12b — Cortex job execution history.

Sending Analyzer Results Back to TheHive

- Cortex sends the analyzer results back to TheHive, where they are automatically attached to the corresponding case as enrichment data. This allows analysts to review threat intelligence details (such as IP reputation, file hash analysis, or URL categorization) directly within the TheHive interface, enabling faster triage and informed decision-making.

Data flow: TheHive → Cortex (analysis request) → Cortex → TheHive (enrichment results)

FLAGS	DATA TYPE	VALUE/FILENAME	DATES
TLP-AMBER	hash	ffb055b6aea06076f2e25772f3f0655e	S: 07/11/2025 17:14
PAP-AMBER	None	VTGetReport: "0 contacted domain"	C: 07/11/2025 17:14
TLP-AMBER	ip	192[.]168[.]192[.]204	S: 07/11/2025 15:43
PAP-AMBER	None	VTGetReport: "0/95"	C: 07/11/2025 15:43
TLP-AMBER	ip	192[.]168[.]192[.]206	S: 07/11/2025 15:43
PAP-AMBER	None	VTGetReport: "0/95"	C: 07/11/2025 15:43

Figure 13 — Cortex sends analyzer results back to TheHive, attached to the case observables.

MISP Integration

- MISP (Malware Information Sharing Platform) was integrated into the SOAR stack as the threat-intelligence backbone for Cortex. MISP stores and correlates Indicators of Compromise (IOCs) — IP addresses, domains, hashes, and URLs — across historical events and external OSINT feeds, and Cortex queries MISP on behalf of TheHive whenever an analyst runs enrichment on an observable. This closes the loop between detection (Wazuh/Suricata), case management (TheHive), and threat intelligence (MISP), allowing analysts to instantly see whether an observable has been seen before, in what campaign, and from which intelligence source — without ever leaving TheHive.
- The sub-sections below walk through the full MISP lifecycle used in this deployment: standing up the platform, the first-run configuration, populating it with threat data (both from a self-managed event catalogue and from subscribed OSINT feeds), and finally wiring it into Cortex so that every observable investigated in TheHive is automatically checked against it.

MISP Deployment :

- MISP was deployed using the coolacid/misp-docker container stack alongside the existing TheHive and Cortex containers, running on the same Docker host.
- The stack includes the MISP core container (soar-misp), a dedicated Redis instance (misp_redis), a MySQL database (thehive-cortex-misp mysql-1), and the MISP modules container (thehive-cortex-misp-modules-1) that provides enrichment/import modules used by MISP itself.
- The MISP web application is published on ports 80 and 443, alongside TheHive (port 9000) and Cortex (port 9001) on the same management LAN.

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
cc02d5863242	redis:6	misp_redis	"docker-entrypoint.s..."	8 months ago	Up 6 days	6379/tcp
5ba3daf20aa7	strangebee/thehive:5.2	thehive-1	"/opt/thehive/entryp..."	8 months ago	Up 6 days	0.0.0.0:9000->9000/tcp
c53c0f152883	coolacid/misp-docker:modules-latest	thehive-cortex-misp-modules-1	"/usr/local/bin/misp..."	8 months ago	Up 6 days	
6c0edcc7fb71	thehiveproject/cortex:latest	soar-cortex-1	"/opt/cortex/entrypo..."	8 months ago	Up 5 days	0.0.0.0:9001->9001/tcp
da3fab504558	coolacid/misp-docker:core-latest	soar-misp	"/entrypoint.sh"	8 months ago	Up 6 days	0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp
2b5f23713bdd	Cassandra:4	thehive-cortex-cassandra-1	"docker-entrypoint.s..."	8 months ago	Up 6 days	7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9042->9042/tcp
679f08b736b4	docker.elastic.co/elasticsearch/elasticsearch:7.17.9	thehive-cortex-elasticsearch-1	"/bin/tini -- /usr/l..."	8 months ago	Up 6 days	0.0.0.0:9200->9200/tcp, 9308/tcp
a0a144c3db56	quay.io/minio/minio	thehive-cortex-minio-1	"/usr/bin/docker-ent..."	8 months ago	Up 6 days	9000/tcp, 0.0.0.0:9002->9002/tcp
58d296469f22	redis:latest	thehive-cortex-redis-1	"docker-entrypoint.s..."	8 months ago	Up 6 days	6379/tcp
facl2e21c159	mysql/mysql-server:5.7	thehive-cortex-misp mysql-1	"/entrypoint.sh mysq..."	8 months ago	Up 6 days (healthy)	3306/tcp, 33060/tcp

Figure 14 — Docker container inventory confirming MISP (soar-misp), its Redis and MySQL backing services, and the MISP modules container running alongside TheHive and Cortex.

Initial Setup and Login :

- On first launch, MISP presents an ‘Initial Install, please configure’ notice on its login screen, prompting the administrator to complete first-run setup (base URL, GnuPG/PGP key, and the default admin password change) before the instance is used in production.
- The platform is reached over HTTPS using the self-signed certificate generated at install time; browsers flag this as ‘Not secure’ until an internal CA-signed certificate is installed, which is expected in an isolated lab/management LAN.
- Once configured, the administrator logs in with an organization email and password to reach the MISP dashboard, from which events, feeds, and users are managed.

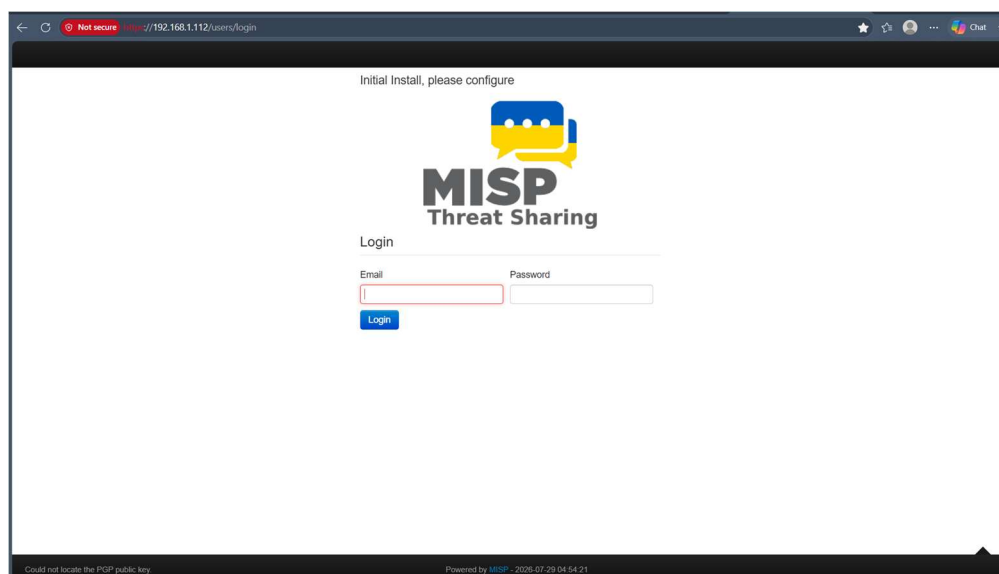


Figure 15 — MISP first-run login screen at <https://192.168.1.112>, showing the ‘Initial Install, please configure’ prompt before the administrator account is finalized.

MISP Web Interface & API Access :

- The MISP web interface is accessible at:
<https://192.168.1.112:80>
- This is the management address for the MISP instance used by Cortex's MISP_2_1 analyzer, and it is also where analysts log in directly to browse events, manage feeds, and add attributes.
- As with TheHive and Cortex, an organization-scoped API key is generated from the MISP user profile (Admin → auth keys); this key authenticates Cortex's requests to MISP's REST API (POST /attributes/restSearch).

Threat Intelligence Feeds :

- MISP's Feeds module (Sync Actions → List Feeds) lets the platform subscribe to and periodically ingest external OSINT and community threat feeds, in addition to internally authored events. Each feed is enabled/disabled individually, cached locally, and can be fetched on demand or on a schedule.
- This deployment has both MISP-format feeds (structured events, e.g. CIRCL OSINT and abuse.ch URLhaus) and freetext/CSV feeds (raw indicator lists, e.g. Tor exit-node lists and IP blocklists) enabled, giving broad indicator coverage across malware, phishing, botnet, and anonymization infrastructure.

Feed	Format	Provider	Coverage
CIRCL OSINT Feed	misp	CIRCL	Curated open-source threat events
The Botvrij.eu Data	misp	Botvrij.eu	Community-shared IOCs
abuse.ch URLhaus	misp	abuse.ch	Malicious URL distribution
ELLIO: IP Feed (Community)	freetext	ellio.tech	Malicious/scanning IPs
blockrules – EmergingThreats	csv	rules.emergingthreats.net	Network blocklist rules
Tor exit / ALL nodes	csv	dan.me.uk	Tor network infrastructure
cybercrime-tracker.net	freetext	cybercrime-tracker.net	C2 panel tracking
Phishtank online valid phishing	csv	Phishtank	Verified phishing URLs
ip-block-list – snort.org	freetext	snort.org	Network IDS blocklist
Feodo IP Blocklist	csv	abuse.ch	Botnet C2 IPs

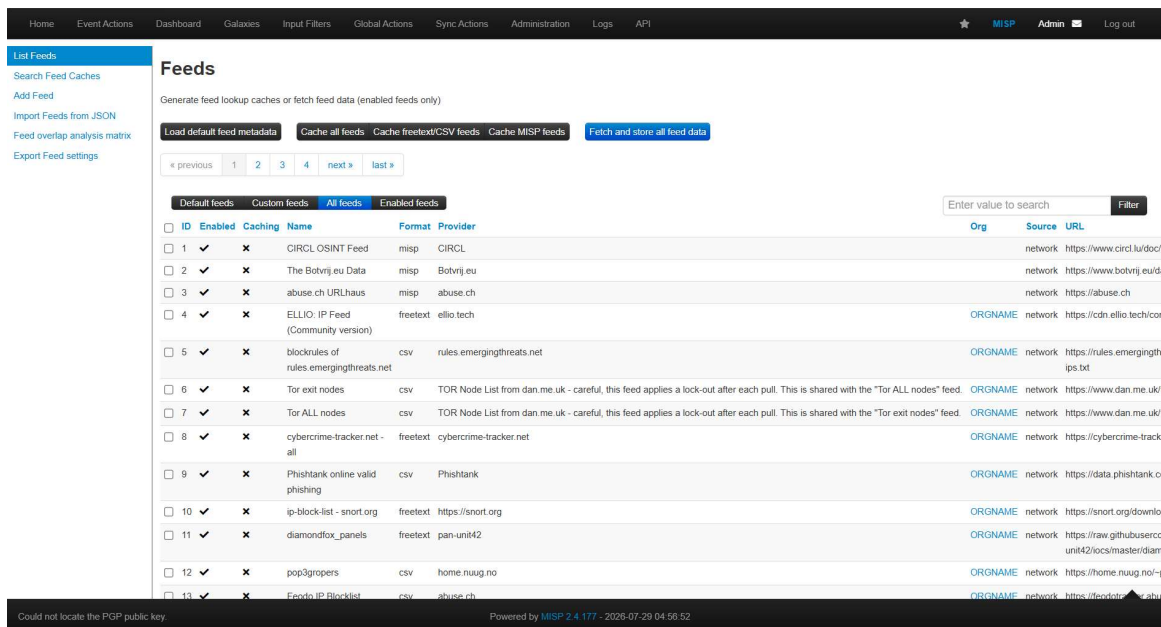


Figure 16 — MISP Feeds management screen (Sync Actions → List Feeds), listing default, custom, and enabled OSINT feeds available for caching and ingestion.

- Administrators can Load default feed metadata, then Cache all feeds / Fetch and store all feed data to pull the latest indicators. Once cached, feed attributes are correlated against event data automatically, so an observable investigated later in TheHive may match a feed-sourced indicator as well as a manually authored MISP event.

Events Repository :

- Every piece of threat intelligence in MISP — whether authored manually or ingested from a feed — is organized as an Event, a container for related attributes (IOCs), context, and tags.
- The List Events view shows, per event, the creator and owner organization, the number of attributes (#Attr.), the number of correlations to other events (#Corr.), and descriptive tags such as `osint:source-type="block-or-filter-list"` that classify the intelligence type.
- Feed-derived events in this deployment (e.g., aggregated IPsum block/filter lists) carry tens of thousands of attributes each and hundreds of correlations, which is exactly the kind of large-scale, continuously updated indicator set that the MISP_2_1 Cortex analyzer draws on when it searches for a match against an observable from TheHive.

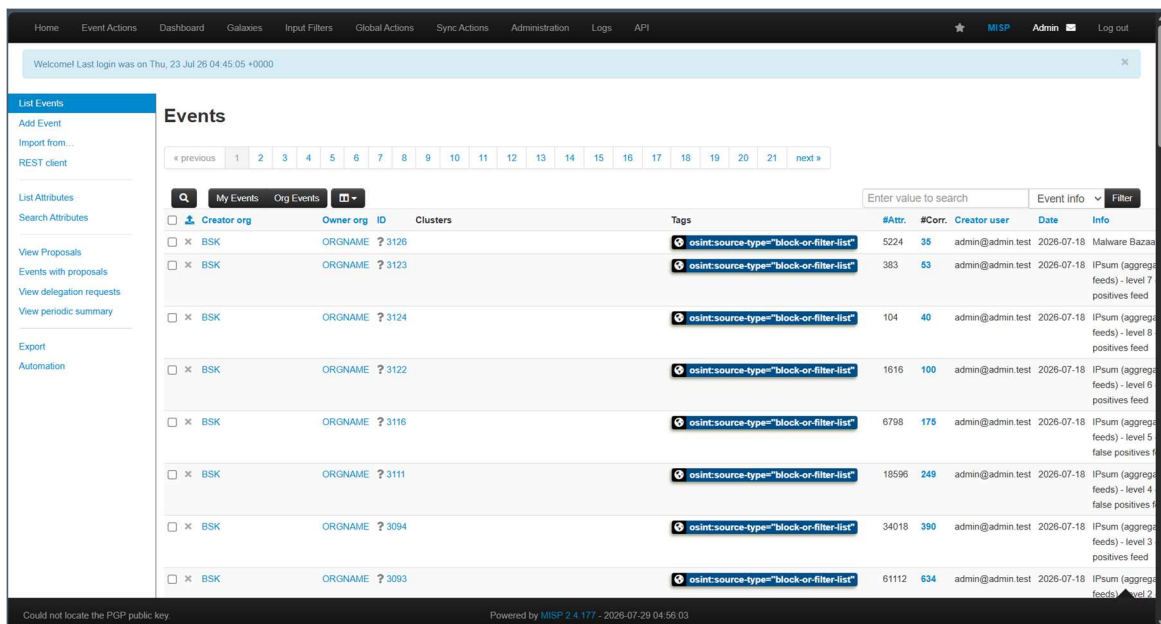


Figure 17 — MISP Events list, showing multiple ingested feed events with attribute and correlation counts and OSINT source-type tagging.

Enabling the MISP Analyzer in Cortex :

- Cortex ships with two MISP-related analyzers: MISPWarningLists_2_0, which checks observables against MISP warning lists to help filter out known false positives, and MISP_2_1, which queries one or more MISP instances for events containing a given observable.
- For this deployment, MISP_2_1 was enabled and configured with a 10-minute cache and TLP:AMBER / PAP:AMBER handling, matching the sensitivity level used across the rest of the SOAR environment.

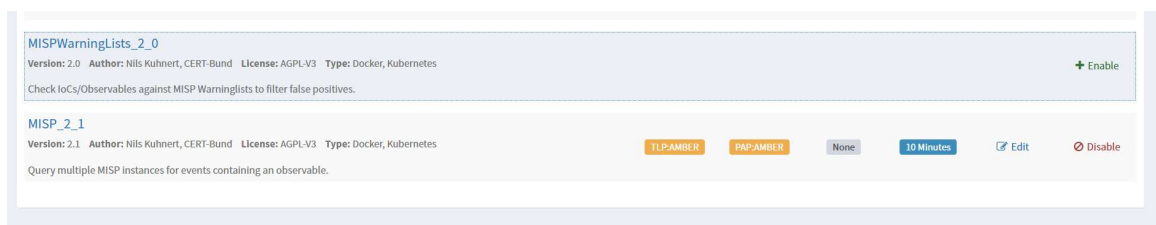


Figure 18 — MISPWarningLists_2_0 and MISP_2_1 analyzers listed in Cortex's Organization → Analyzers configuration.

Configuring the MISP Analyzer :

- Enabling MISP_2_1 opens a configuration form requiring the MISP server details:
 - url — the base URL of the MISP instance: https://192.168.1.112:80
 - key — the MISP API key generated for the Cortex service account.
 - cert_check — whether Cortex should validate the MISP server's TLS certificate (set according to whether MISP is using a trusted or self-signed certificate).
- Once saved, Cortex maintains a persistent connection profile for the MISP server, allowing every subsequent analyzer job to query it automatically.

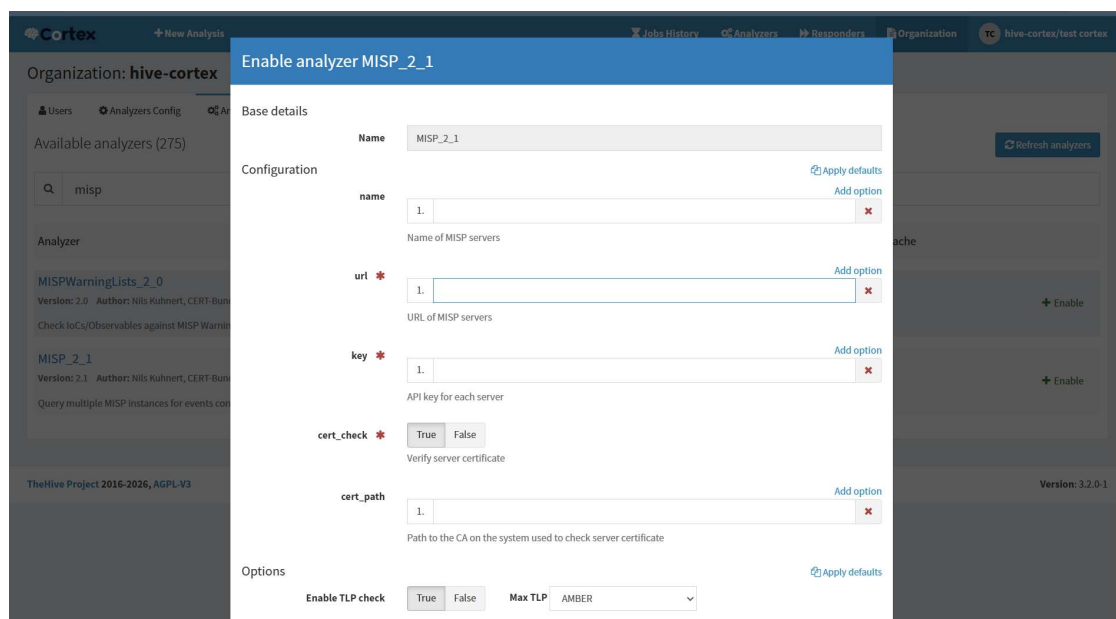


Figure 19 — Cortex 'Enable analyzer MISP_2_1' configuration form, where the MISP server URL and API key are registered.

Adding customized Threat Intelligence Indicators (IOCs) in MISP :

- Analysts populate MISP with known-bad indicators by adding attributes to MISP events. Each attribute has a category and type (e.g., Network activity / ip-dst), a distribution level, and an optional contextual comment.

- Attributes can also be flagged For Intrusion Detection System, marked for batch import, or associated with first-seen / last-seen timestamps for sighting correlation.
- Once an IOC (such as the internal infrastructure DDoS event's source IPs) is stored in MISP, any future observable in TheHive that matches it will be returned by the MISP_2_1 analyzer, giving the analyst immediate historical context.

Figure 20 — MISP 'Add Attribute' dialog used to register new IOCs (IP addresses, hashes, domains, URLs) against a MISP event.

Running the MISP Analyzer on an Observable :

- From a TheHive case, the analyst selects an observable — for example, the source IP of a Suricata ICMP-flood alert — and chooses Run Analyzer.
- Both configured analyzers, MISP_2_1 and VirusTotal_GetReport_3_1, are listed and can be run individually or together, giving reputation data (VirusTotal) and historical event correlation (MISP) side by side.

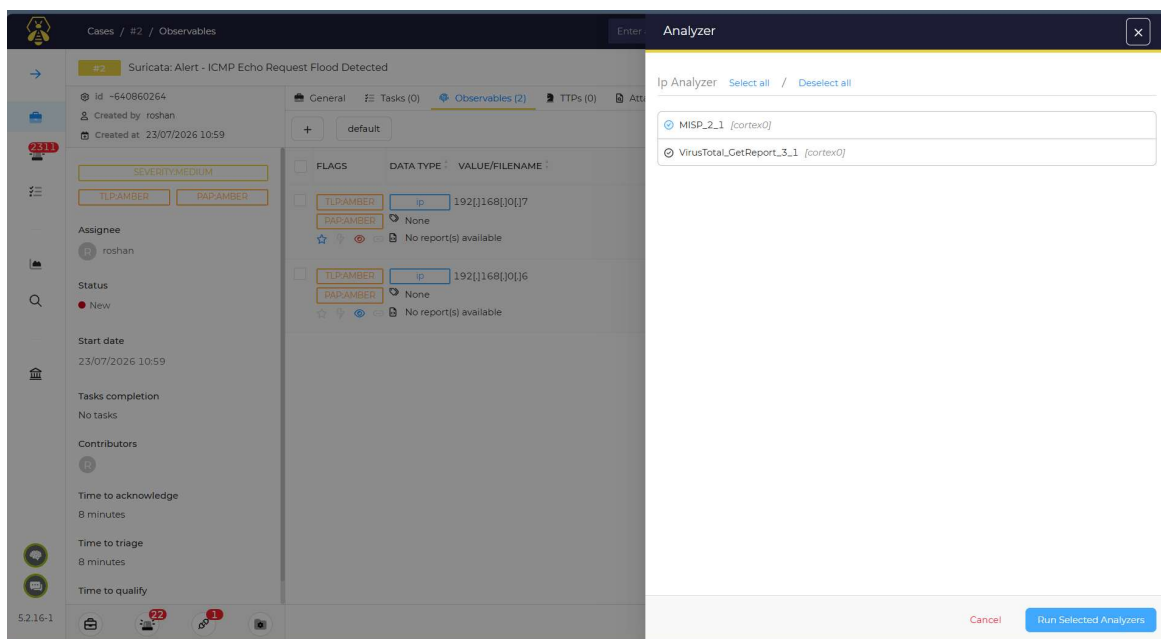


Figure 21 — Running the MISP_2_1 and VirusTotal_GetReport_3_1 analyzers against an observable on a Suricata ICMP Echo Request Flood case in TheHive.

MISP Analysis Results :

- When the job completes, Cortex returns a standardized report showing how many MISP events matched the observable. In this example, the observable 192.168.0.7 matched one prior MISP event.
- The result includes the Event ID, Event Info (a human-readable description of the prior incident), the event UUID, and the originating organization — giving the analyst immediate context on whether the IP has been associated with malicious activity before.

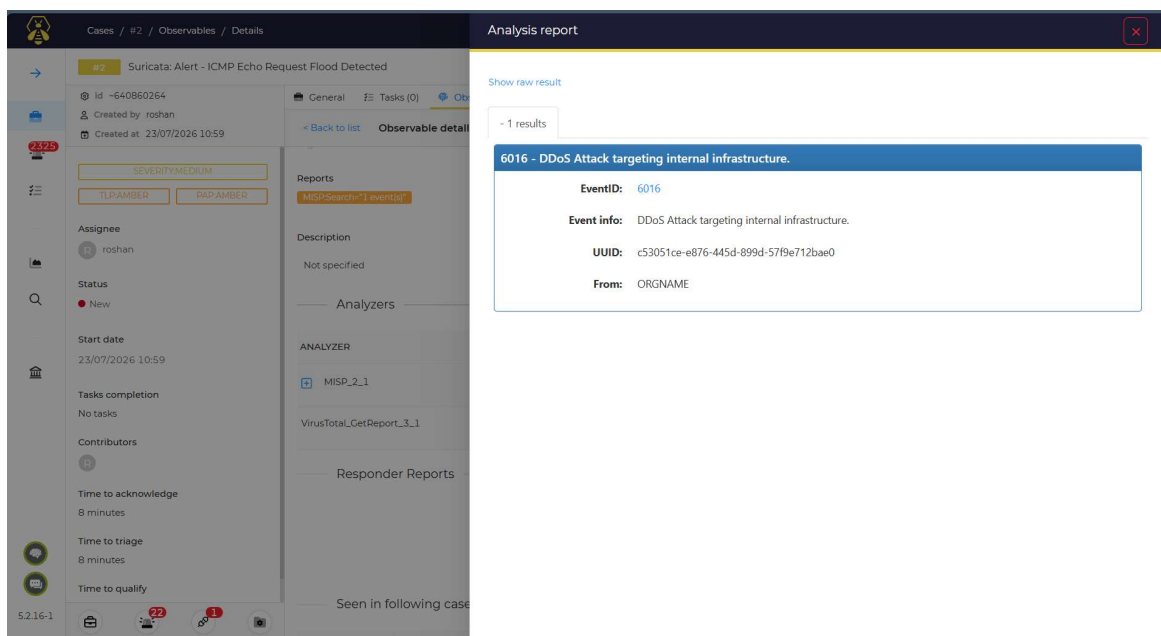


Figure 22 — TheHive analysis report for observable 192.168.0.7, showing a MISP match against Event 6016 – ‘DDoS Attack targeting internal infrastructure.’

- The full Cortex job report (visible under Cortex → Jobs History) contains the raw JSON returned by MISP, including the event date, threat level, TLP marking, and UUID — useful for deeper investigation or export to other tools.

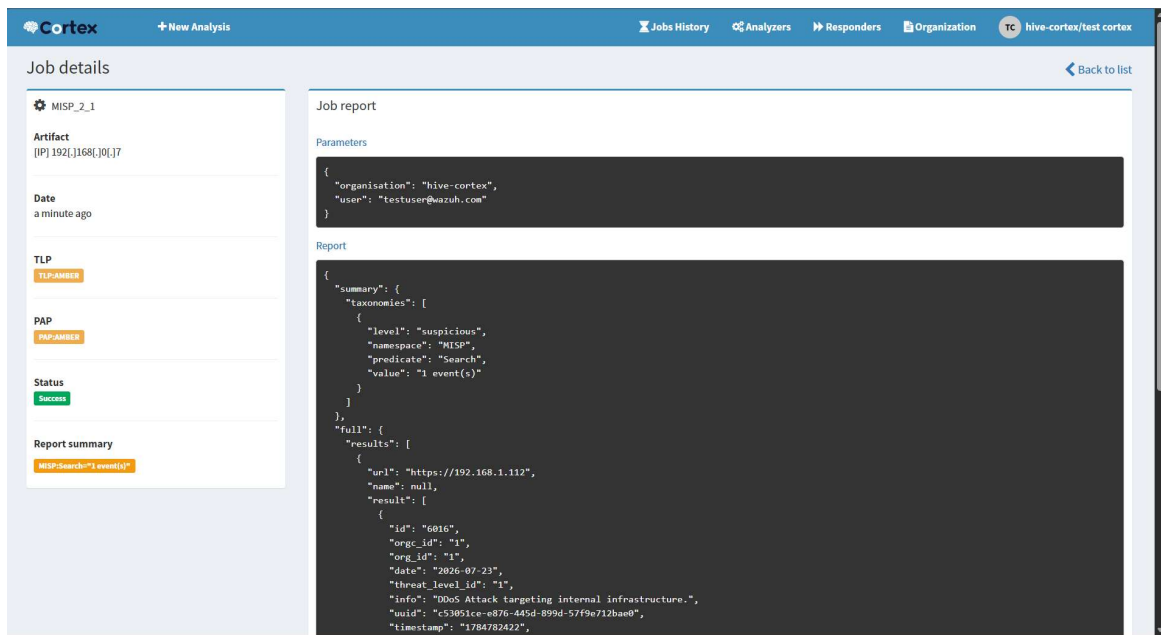


Figure 23 — Cortex MISP_2_1 job details, showing the parameters sent to MISP and the raw enrichment report (Event ID 6016, DDoS Attack targeting internal infrastructure) returned to TheHive.

Value Added by the MISP Integration :

- Historical correlation — observables are automatically checked against previously recorded events without manual lookups.
- Broad, continuously updated coverage — the subscribed OSINT and community feeds (CIRCL, abuse.ch, Botvrij.eu, Emerging Threats, Phishtank, Feodo Tracker, Tor node lists, and others) mean the MISP_2_1 analyzer is checking observables against tens of thousands of externally sourced indicators, not just internally authored events.
- Shared threat context — event metadata (info, UUID, threat level, TLP, organization, correlation count) gives analysts the story behind an IOC, not just a reputation score.
- False-positive reduction — the MISPWarningLists analyzer can flag observables that match known-benign infrastructure lists before an analyst spends time investigating them.
- Closed-loop intelligence — new IOCs discovered during an investigation can be added back into MISP as attributes, growing the organization's own event catalogue for future correlation, independent of the external feeds.

Conclusion

- The integrated SOAR environment successfully brings together Wazuh, Suricata, TheHive, Cortex, and MISP into a unified security operations ecosystem that enhances both visibility and response efficiency.
- Suricata provides real-time network intrusion detection, while Wazuh acts as the central SIEM, aggregating and correlating security events across the infrastructure.
- TheHive enables automated case management and structured incident handling, and Cortex strengthens the workflow by enriching alerts with powerful threat intelligence and analysis capabilities.
- MISP extends this further by providing a shared, correlatable repository of Indicators of Compromise, so that every new observable is automatically checked against the organization's own threat history — not just external reputation feeds.
- This architecture not only automates alert triage and reduces analyst workload but also minimizes response time through seamless integration between detection, analysis, threat intelligence, and orchestration layers.
- By aligning with modern SOC best practices, the setup establishes a scalable, open-source, and cost-effective foundation for proactive threat detection, investigation, and response automation.